Barème détaillé — Projet noté — Site vitrine d'un événement

> Document commun **enseignant et étudiants**. Il détaille le barème /20 du plan de module sans en changer la répartition : **18 points de groupe** (critères A à G) + **2 points individuels** (critère H, soutenance) = **20 points**. La grille de saisie correspondante est `Grille_notation.xlsx` (mêmes codes de sous-critères).

1. Vue d'ensemble

Critère	Sous-critères	Points
A. Architecture & qualité du code	A1 Layers (1) · A2 Tokens (1) · A3 Nesting & organisation (1) · A4 Validité & commentaires (1)	**4**
B. Mise en page	B1 `grid-template-areas` (1) · B2 `subgrid` (1) · B3 Flexbox & alignements (1)	**3**
C. Responsive & container queries	C1 Mobile-first & typographie fluide (1) · C2 Container query (1) · C3 Robustesse à 320 px (1)	**3**
D. Sélecteurs modernes & thème clair/sombre	D1 `:has()`, `:is()`, `:where()` (1) · D2 Thème clair/sombre (1)	**2**
E. Animations & interactions	E1 Transitions (1) · E2 `@keyframes` (1) · E3 `prefers-reduced-motion` (1)	**3**
F. Accessibilité	F1 Sémantique & alternatives textuelles (1) · F2 Focus & contrastes (1)	**2**
G. Qualité visuelle & finition	G1 Identité & cohérence visuelle (0,5) · G2 Complétude & finition (0,5)	**1**
Sous-total groupe (A → G)		**18**
H. Soutenance — note individuelle	H1 Localiser & expliquer (1) · H2 Justifier & faire varier (1)	**2**
Total		**20**
Bonus technique (plafonné)	Fonctionnalité avancée maîtrisée	**+1 max**
Pénalités	Framework CSS / JS non autorisé −3 · Retard −1 par tranche de 15 min · README de localisation absent −1	

Comment lire les niveaux

Chaque sous-critère est noté sur trois niveaux observables :

Niveau	Sous-critère sur 1 pt	Sous-critère sur 0,5 pt	Signification
0	0	0	Absent, faux, ou présent mais non fonctionnel.

Niveau	Sous-critère sur 1 pt	Sous-critère sur 0,5 pt	Signification
Partiel	0,5	0,25	Présent et fonctionnel, mais incomplet, peu pertinent ou appliqué de façon incohérente.
Complet	1	0,5	Présent, fonctionnel, pertinent, appliqué de façon cohérente **et localisé dans le README**.

> **Règle de localisation** : une exigence que l'enseignant ne retrouve pas à l'emplacement indiqué dans le README (fichier + ligne) ne peut pas dépasser le niveau « partiel ». Une exigence non localisée **et** introuvable en 2 minutes est notée 0.

---

2. Descripteurs par sous-critère (critères A à G — note de groupe)

A. Architecture & qualité du code — 4 pts

Sous-critère	0	Partiel	Complet
A1 — Layers (1 pt)	Pas de `@layer`, ou layers déclarés mais vides / non utilisés.	L'ordre `@layer reset, base, layout, components, utilities;` est déclaré, mais des règles importantes sont hors layer sans raison, ou rangées dans le mauvais layer (ex. un composant `.card` dans `base`).	Ordre déclaré **une seule fois en tête** de `main.css` ; chaque fichier enveloppé dans son layer ; chaque règle est dans le layer qui correspond à son rôle ; les utilitaires gagnent sans `!important` grâce à l'ordre des layers. Visible dans DevTools (onglet Styles, regroupement par layer).
A2 — Tokens (1 pt)	Pas de custom properties, ou seulement 1–2 variables isolées ; valeurs « magiques » partout.	Tokens présents mais à un seul niveau (ex. `--violet: #7a3cff` utilisé directement dans les composants), ou nombreuses valeurs codées en dur restantes (couleurs, espacements).	Deux niveaux : **primitifs** (`--color-violet-600`, `--space-4`…) et **sémantiques** (`--color-surface`, `--color-accent`, `--space-section`…). Les composants n'utilisent que les sémantiques. Au plus quelques valeurs littérales justifiées (ex. `0`, `1px` de bordure).
A3 — Nesting & organisation (1 pt)	Pas de nesting natif, ou CSS en un seul bloc sans structure ; nommage incohérent (`.box2`, `.bleu`, `#truc`).	Nesting présent mais excessif (≥ 4 niveaux, sélecteurs très spécifiques) ou ponctuel ; nommage partiellement cohérent.	Nesting natif lisible (≤ 3 niveaux, `&:hover`, `&::before`, `@media` / `@container` imbriqués dans le composant) ; une **convention de nommage annoncée** dans le README (BEM, ou `composant__élément--variante`, ou autre) et respectée ; fichiers découpés par layer.
A4 — Validité & commentaires (1 pt)	Erreurs de syntaxe réelles (accolade manquante, propriété inventée), ou `!important` utilisé pour « forcer » des styles.	Code valide mais peu ou pas commenté, **ou** quelques `!important` non justifiés hors utilitaires.	Validateur W3C sans erreur réelle (les avertissements sur des syntaxes récentes que le validateur ne connaît pas encore sont listés dans le README et ne pénalisent pas) ; commentaires utiles (rôle de chaque fichier, sections, « pourquoi » des choix non évidents) ; aucun `!important` hors utilitaires justifiés.

B. Mise en page — 3 pts

Sous-critère	0	Partiel	Complet
B1 — `grid-template-areas` (1 pt)	Aucune grille à zones nommées.	Zones nommées présentes mais figées (même disposition sur mobile et desktop) ou utilisées sur un cas trivial (une seule zone).	Au moins une grille à zones nommées **lisible** (≥ 3 zones) qui se **réorganise** entre mobile et desktop en ne changeant que `grid-template-areas` / `grid-template-columns` (ex. programme, hero ou page entière).
B2 — `subgrid` (1 pt)	Aucun `subgrid`.	`subgrid` présent mais sans effet visible (ex. l'enfant n'occupe pas plusieurs pistes du parent).	`subgrid` pertinent : par ex. cartes intervenants dont titres, textes et boutons sont **alignés d'une carte à l'autre** (`grid-row: span 3; grid-template-rows: subgrid;`), ou créneaux du programme alignés sur les colonnes du parent. L'effet est démontrable en ajoutant du texte à une seule carte.
B3 — Flexbox & alignements (1 pt)	Mise en page des composants par `float`, `position: absolute` ou marges négatives « au pixel ».	Flexbox utilisé mais alignements approximatifs (éléments décalés, `margin-left: 37px`), `gap` remplacé par des marges.	Flexbox pour les composants (navigation, groupes de boutons, en-tête de carte, footer…) avec `gap`, `flex-wrap`, raccourci `flex` ; alignements propres et réguliers ; choix Grid vs Flexbox cohérent (2D → Grid, 1D → Flexbox).

C. Responsive & container queries — 3 pts

Sous-critère	0	Partiel	Complet
C1 — Mobile-first & typographie fluide (1 pt)	Desktop-first (`max-width` partout) et tailles fixes en px.	Mobile-first **ou** `clamp()`, mais pas les deux ; ou `clamp()` appliqué à un seul titre.	Styles de base pour petit écran, `@media (width >= …)` / `min-width` pour élargir ; échelle typographique fluide en `clamp()` (au moins titres + corps ou espacements de section), de préférence dans les tokens.
C2 — Container query (1 pt)	Pas de `@container`.	`container-type` + `@container` présents mais le composant n'est utilisé que dans un seul contexte (la CQ se comporte comme une media query), ou l'effet est minime.	Au moins **un composant piloté par container query** (ex. carte intervenant horizontale quand son conteneur dépasse 30rem), placé dans **deux contextes de largeurs différentes** de la page ; nom de conteneur ou unités `cqi` utilisés à bon escient.
C3 — Robustesse à 320 px (1 pt)	Défilement horizontal à 320 px, contenu coupé ou superposé.	Pas de défilement horizontal, mais des éléments mal lisibles (texte qui déborde d'un bouton, tableau écrasé, image déformée).	À 320 px : **aucun défilement horizontal** (vérifié DevTools), images fluides (`max-inline-size: 100%`, `aspect-ratio` / `object-fit`), mots longs gérés (`overflow-wrap`), lisible aussi à 1440 px et au zoom 200 %.

D. Sélecteurs modernes & thème clair/sombre — 2 pts

Sous-critère	0	Partiel	Complet
D1 — `:has()`, `:is()`, `:where()` (1 pt)	Aucun de ces sélecteurs.	Un seul des deux usages (`:has()` **ou** `:is()` / `:where()`), ou `:has()` utilisé là où un sélecteur simple suffisait (ex. `.card:has(.card__title)` alors que toutes les cartes ont un titre).	`:has()` qui **apporte quelque chose** (ex. carte mise en avant si elle contient un badge, formulaire en erreur, bascule de thème, grille qui change si elle contient plus de N éléments) **et** `:is()` / `:where()` pour factoriser ou abaisser la spécificité (ex. `:where(h1, h2, h3)` dans `base`). L'étudiant sait dire la différence de spécificité entre les deux.
D2 — Thème clair/sombre (1 pt)	Un seul thème.	Thème sombre présent mais incomplet : contrastes insuffisants dans un des thèmes, éléments restés « clairs », `color-scheme` absent (barres de défilement, champs de formulaire non adaptés).	`color-scheme: light dark` + `light-dark()` dans les tokens sémantiques **ou** `@media (prefers-color-scheme: dark)` ; les deux thèmes sont complets et lisibles. (La bascule CSS-only avec `:has()` n'est pas exigée ici : elle peut compter dans le bonus technique.)

E. Animations & interactions — 3 pts

Sous-critère	0	Partiel	Complet
E1 — Transitions (1 pt)	Aucune transition, ou une seule.	Au moins 2 transitions, mais `transition: all`, propriétés coûteuses (`width`, `top`, `margin`) ou durées incohérentes.	**Au moins 2 transitions** utiles (survol/focus de cartes, boutons, ouverture de FAQ…) ciblant des propriétés précises, de préférence `transform` / `opacity` /couleurs ; durées et courbes issues des tokens (`--duration-fast`, `--ease-out`).
E2 — `@keyframes` (1 pt)	Aucun `@keyframes`.	`@keyframes` présent mais décoratif et gênant (clignotement permanent, animation infinie qui distrait la lecture) ou non utilisé.	Au moins **un `@keyframes` utilisé** de façon pertinente (apparition du hero, pulsation du CTA limitée en nombre d'itérations, compte à rebours…), animant de préférence `transform` / `opacity`.
E3 — `prefers-reduced-motion` (1 pt)	Aucun traitement.	Traitement partiel : seules certaines animations sont neutralisées, ou neutralisation brutale qui casse un état (élément resté invisible à `opacity: 0`).	Toutes les animations et transitions de mouvement sont **supprimées ou réduites** sous `@media (prefers-reduced-motion: reduce)` (ou animations écrites seulement dans `no-preference`), sans perte de contenu. Vérifiable via DevTools > Rendu > Émuler `prefers-reduced-motion`.

F. Accessibilité — 2 pts

Sous-critère	0	Partiel	Complet
F1 — Sémantique & alternatives textuelles (1 pt)	« Soupe de `div` », pas de `lang`, images sans `alt`.	Landmarks présents mais hiérarchie de titres cassée (saut `h1` → `h4`, plusieurs `h1`), ou `alt` absents / non pertinents (`alt="image"`) sur une partie des images.	`lang="fr"`, `header` / `nav` / `main` / `footer`, sections titrées, **un seul `h1`** et hiérarchie continue ; listes pour les listes ; FAQ en `<details>/<summary>` ; boutons = `<button>`, liens = `<a href>` ; `alt` descriptif pour les images informatives, `alt=""` pour les décoratives.

Sous-critère	0	Partiel	Complet
F2 — Focus & contrastes (1 pt)	`outline: none` sans remplacement, ou contrastes illisibles.	`:focus-visible` présent sur une partie des éléments seulement, ou un contraste AA non respecté (souvent dans le thème sombre ou sur les boutons).	`:focus-visible` **visible sur tous les éléments interactifs** (navigation au clavier possible du début à la fin, ordre logique) ; contrastes **AA** dans les deux thèmes : 4,5:1 pour le texte courant, 3:1 pour le grand texte et les composants d'interface ; cibles tactiles d'au moins 24 × 24 px.

G. Qualité visuelle & finition — 1 pt

Sous-critère	0	Partiel	Complet
G1 — Identité & cohérence visuelle (0,5 pt)	Aucune direction visuelle, styles navigateur par défaut.	Une direction visuelle existe mais n'est pas tenue (3 polices, couleurs hors palette, espacements irréguliers).	Identité claire et adaptée au thème choisi (palette, typographie, iconographie), appliquée de façon homogène sur toute la page.
G2 — Complétude & finition (0,5 pt)	Au moins deux sections attendues manquantes.	Toutes les sections sont là, mais certaines sont vides, en « lorem ipsum », ou présentent des défauts visibles (image cassée, lien mort).	Les 6 sections (hero, programme, intervenants, billetterie, FAQ, footer) sont complètes avec un contenu fictif réaliste ; aucun défaut visible ; crédits des images dans le footer.

---

3. Bonus technique et pénalités

Bonus technique — +1 point maximum (appliqué à la note de groupe)

Élément	Valeur indicative
Animation pilotée par le scroll (`animation-timeline: view()` / `scroll()`) **dans un** `@supports`, avec rendu correct sans (Firefox ne la prend pas en charge)	+0,5 à +1
View Transitions (same-document) maîtrisées, avec repli sans erreur	+0,5 à +1
Anchor positioning, en amélioration progressive (`@supports (anchor-name: --x)`)	+0,5 à +1
`@property` animé (ex. dégradé ou compteur animé)	+0,5
`@starting-style` / Popover API / `<dialog>` animés proprement	+0,5
Bascule de thème CSS-only via `:has()` (exigence 5, bonus technique)	+0,5

- « **Maîtrisée** » = la fonctionnalité fonctionne, est **expliquée** dans le README et le membre interrogé sait l'expliquer. Une fonctionnalité recopiée et non comprise ne rapporte rien.
- Le total du bonus technique est **plafonné à +1**, quel que soit le nombre de fonctionnalités.

Pénalités (appliquées à la note de groupe)

Pénalité	Valeur	Constat
Framework CSS (Bootstrap, Tailwind…), préprocesseur (Sass, Less), bibliothèque JS, ou JavaScript au-delà de 20 lignes / non justifié	**−3**	Lecture du code et du dossier rendu.

Pénalité	Valeur	Constat
Retard de rendu (heure limite : J3 14h30)	**−1 par tranche de 15 min commencée**	Horodatage du dernier commit / du dépôt de l'archive. 14h31 → −1 ; 14h46 → −2.
README de localisation des exigences absent	**−1**	Fichier `README.md` absent ou vide.

4. Calcul des notes

```
Note de groupe  = MIN(18 ; MAX(0 ; (A + B + C + D + E + F + G) + bonus technique − pénalités))
Note finale     = MIN(20 ; MAX(0 ; note de groupe + H individuel + modulation))
```

- La **note de groupe** (sur 18) est commune aux trois membres. Elle est **plafonnée à 18 avant l'ajout de H** : le bonus technique sert à compenser des points perdus, il ne permet pas de dépasser 18.
- **H** (sur 2) est **individuel** : chaque membre répond à sa propre question pendant la soutenance.
- **Modulation individuelle (−2 à +2)** : selon la contribution réelle observée (historique Git, observation pendant les sprints, stand-up, revue croisée). Par défaut 0. Toute modulation non nulle est justifiée par écrit dans la grille (feuille « Individuel »).
- La note finale est bornée entre 0 et 20.

5. Critère H — Soutenance individuelle (2 pts)

Déroulé

Soutenance de **7 min par groupe** : 4 min de démonstration, puis **3 min de questions individuelles** (≈ 1 min par membre). L'enseignant choisit pour chaque membre **une question différente** dans la banque ci-dessous, appliquée **au code du groupe** (pas forcément à la partie écrite par ce membre : chaque membre doit pouvoir expliquer chaque ligne). L'étudiant peut montrer son code à l'écran.

Sous-critère	0	Partiel	Complet
H1 — Localiser & expliquer (1 pt)	Ne retrouve pas le code concerné ou explique de façon erronée.	Retrouve le code mais l'explication est approximative ou récitée.	Retrouve rapidement le code et explique **ce qu'il fait et comment** avec le bon vocabulaire.
H2 — Justifier & faire varier (1 pt)	Ne sait pas pourquoi ce choix a été fait.	Justifie le choix mais ne sait pas prédire l'effet d'une modification.	Justifie le choix (alternative écartée) **et** prédit correctement l'effet d'une variation proposée par l'enseignant.

Lecture des réponses de la banque : **2 pts** = H1 + H2 complets · **1 pt** = explication correcte mais justification ou prédiction manquante · **0** = réponse fausse ou absente.

Banque de 20 questions individuelles

Cascade & layers

#	Question	2 pts	1 pt	0
Q1	Montrez la déclaration de vos layers. Pourquoi un sélecteur `.u-hidden` dans `utilities` l'emporte-t-il sur `#programme .card` dans `components` ?	Le **layer** est comparé **avant la spécificité** : le dernier layer déclaré gagne pour les déclarations normales, quelle que soit la spécificité. Sait que l'ordre est fixé par la première déclaration `@layer reset, base, …;` et ce qui se passerait en inversant l'ordre.	Sait que « l'ordre des layers compte » sans savoir qu'il prime sur la spécificité.	Invoque la spécificité ou l'ordre d'apparition dans le fichier.
Q2	Une règle écrite **hors de tout layer** et une règle dans `utilities` ciblent le même élément : laquelle gagne, et pourquoi ?	Les styles **hors layer** (layer implicite final) l'emportent sur tous les layers pour les déclarations normales ; c'est pourquoi tout le code du groupe est dans un layer. Réponse approfondie : avec `!important`, l'ordre s'inverse.	Donne la bonne réponse sans l'expliquer.	Répond que `utilities` gagne.
Q3	Expliquez un usage de `inherit`, `initial`, `unset`, `revert` ou `revert-layer` dans votre code (ou dites où vous pourriez en utiliser un).	Définit précisément le mot-clé choisi (ex. `revert-layer` = revient à la valeur du layer précédent) et propose un cas concret dans leur code.	Définition correcte mais aucun cas d'usage.	Confond les mots-clés.

Tokens & custom properties

#	Question	2 pts	1 pt	0
Q4	Montrez un token primitif et le token sémantique qui l'utilise. Pourquoi deux niveaux ?	Primitif = valeur brute de la palette ; sémantique = rôle (`--color-surface`). Changer de thème ou de marque = modifier les sémantiques sans toucher aux composants. Sait indiquer quels fichiers changeraient pour un 2e thème.	Distingue les deux niveaux sans en donner l'intérêt.	Ne fait pas la différence.
Q5	Que se passe-t-il si un composant utilise `var(--color-accent, orange)` et que `--color-accent` n'est pas défini ? Et s'il est défini avec une valeur invalide pour `color` ?	Non défini → valeur de repli `orange`. Valeur invalide → la déclaration devient « invalide au moment du calcul » : la propriété prend sa valeur héritée ou initiale, **pas** le repli.	Connaît le repli mais pas le cas de la valeur invalide.	Pense que la déclaration est ignorée et que la règle précédente s'applique, sans autre nuance, ou ne sait pas.

Nesting & sélecteurs modernes

#	Question	2 pts	1 pt	0
Q6	Lisez ce bloc imbriqué de votre code et écrivez le sélecteur « à plat » équivalent. Quel est le rôle du `&` ?	Donne le sélecteur exact ; sait que `&` représente le sélecteur parent (et que `&:hover` ≠ `& :hover`).	Sélecteur à plat correct mais rôle du `&` flou.	Sélecteur faux.

#	Question	2 pts	1 pt	0
Q7	Montrez votre usage de `:has()`. Que fait-il, et comment l'auriez-vous fait sans `:has()` ?	Explique le sélecteur « parent/relationnel » ; sans `:has()` il aurait fallu une classe ajoutée à la main ou du JavaScript. Prédit l'effet si l'élément ciblé disparaît du HTML.	Explique ce que fait la règle sans l'alternative.	Ne sait pas expliquer la règle.
Q8	Quelle est la différence entre `:is(h2, h3)` et `:where(h2, h3)` ? Pourquoi avoir choisi l'un plutôt que l'autre à cet endroit ?	`:is()` prend la spécificité de son argument le plus spécifique ; `:where()` a une spécificité **nulle** (facile à surcharger, idéal dans `reset` / `base`). Justifie le choix dans leur code.	Connaît la différence de spécificité sans justifier le choix.	Pense qu'ils sont identiques.

Grid, subgrid & Flexbox

#	Question	2 pts	1 pt	0
Q9	Montrez votre `grid-template-areas`. Comment passez-vous de la disposition mobile à la disposition desktop ?	Seules les déclarations de la grille changent dans la media query, les enfants gardent leur `grid-area` ; sait que chaque zone doit former un rectangle et ce que signifie un `.`.	Explique le changement sans connaître les règles (rectangle, `.`).	Ne retrouve pas la grille ou ne sait pas l'expliquer.
Q10	Montrez votre `subgrid`. Que se passe-t-il visuellement si on le retire ?	Sans subgrid, chaque carte a ses propres lignes : les titres/boutons ne sont plus alignés d'une carte à l'autre ; sait que l'enfant doit couvrir plusieurs pistes (`span`). Peut le démontrer en commentant la ligne.	Sait que « ça aligne » sans pouvoir dire quoi ni pourquoi.	Ne sait pas.
Q11	Expliquez `repeat(auto-fit, minmax(16rem, 1fr))` (ou l'équivalent dans votre code). Différence avec `auto-fill` ?	Crée autant de colonnes d'au moins 16rem que possible, étirées à `1fr` ; `auto-fit` replie les pistes vides (les éléments s'étirent), `auto-fill` les conserve.	Explique la formule sans la différence fit/fill.	Ne sait pas.
Q12	Pourquoi avoir utilisé Flexbox (et non Grid) pour tel composant ? Que fait `flex: 1 1 12rem` ?	Flexbox = distribution sur un axe, contenu qui dicte la taille ; décompose `flex-grow / flex-shrink / flex-basis`.	Justification correcte mais raccourci `flex` mal décomposé (ou l'inverse).	Ni l'un ni l'autre.

Responsive & container queries

#	Question	2 pts	1 pt	0
Q13	Montrez votre container query. Quelle différence avec une media query ? Où le composant est-il utilisé dans deux contextes ?	La CQ réagit à la taille **du conteneur** (`container-type: inline-size`), pas de la fenêtre ; montre les deux contextes et l'effet ; sait qu'un élément ne peut pas interroger sa propre taille.	Différence correcte sans démonstration sur leur page.	Confond avec une media query.
Q14	Décomposez votre `clamp()` de typographie fluide. Que vaut la taille sur un écran de 320 px ?	Minimum / valeur préférée (souvent `rem + vw`) / maximum ; calcule ou estime la valeur à 320 px ; sait pourquoi on garde une part en `rem` (respect du zoom).	Explique les trois arguments sans savoir estimer.	Ne sait pas.

#	Question	2 pts	1 pt	0
Q15	Comment avez-vous vérifié l'absence de défilement horizontal à 320 px ? Quelle propriété a corrigé le problème le plus gênant ?	Méthode (mode responsive DevTools, 320 px, recherche de l'élément fautif) + correctif précis (`min-inline-size: 0` , `overflow-wrap: anywhere` , `max-inline-size: 100%` ...) expliqué.	Méthode sans correctif expliqué.	Pas de méthode.

Couleurs & thème

#	Question	2 pts	1 pt	0
Q16	Comment fonctionne votre thème sombre ? Rôle de `color-scheme` ?	`light-dark(a, b)` choisit la valeur selon le `color-scheme` calculé ; sans `color-scheme: light dark` , `light-dark()` ne bascule pas et les éléments natifs (champs, barres de défilement) ne s'adaptent pas. Explique la bascule CSS-only s'il y en a une.	Décrit le mécanisme sans le rôle de `color-scheme` .	Ne sait pas.
Q17	Pourquoi `oklch()` (ou `color-mix()`) plutôt qu'un hexadécimal à cet endroit ? Comment avez-vous vérifié les contrastes ?	Luminosité perçue homogène (`L`), variantes faciles à dériver ; vérification avec un outil (DevTools, contrast checker) avec les seuils 4,5:1 / 3:1.	Un des deux volets seulement.	Ni l'un ni l'autre.

Animations & accessibilité

#	Question	2 pts	1 pt	0
Q18	Pourquoi animer `transform` / `opacity` plutôt que `top` / `width` ? Montrez une de vos transitions.	Ces propriétés n'entraînent ni recalcul de mise en page ni repeinture et sont gérées par le compositeur → animation fluide ; justifie la durée et la courbe choisies.	Sait que « c'est plus performant » sans dire pourquoi.	Ne sait pas.
Q19	Montrez votre gestion de `prefers-reduced-motion` . Comment la tester ? Qu'advient-il de votre `@keyframes` ?	Montre la media query, explique l'effet sur chaque animation (aucun contenu perdu) et la teste en direct via l'émulation DevTools.	Montre la règle sans savoir la tester (ou l'inverse).	Absente ou inconnue.
Q20	Naviguez au clavier sur votre page. Pourquoi `:focus-visible` plutôt que `:focus` ? Comment est construite votre FAQ ?	`:focus-visible` n'affiche l'anneau que lorsque le navigateur juge le focus utile (clavier), sans le supprimer pour les utilisateurs clavier ; FAQ en `<details>/<summary>` = accessible et sans JavaScript ; la navigation clavier est démontrée.	Explique `:focus-visible` mais pas la sémantique de la FAQ (ou l'inverse).	Ne sait pas.

---

6. Vérification du total

	Points
A (4) + B (3) + C (3) + D (2) + E (3) + F (2) + G (1)	**18** (note de groupe)
H (2)	**2** (note individuelle)

	Points
Total	**20**